



Algoritmos y estructuras de datos.

**Trabajo práctico N°1.
Informe Proyecto 3-Algoritmos de Ordenamiento**

Integrantes del grupo: - Hasenauer, Fernanda.
-Hernandez, Tomas
-Leguizamon, Augusto.

Fecha de entrega: 28/9/26.

Link del repositorio:

[AyED2026c2-Hernandez-Hasenauer-Leguizamon/TrabajoPractico_1/proyecto_3 at main · Hernandez-TV/AyED2026c2-Hernandez-Hasenauer-Leguizamon](#)

A) Análisis y planteo.

Se estudiaron tres algoritmos de ordenamiento con características diferentes: Bubble Sort, QuickSort, Radix Sort.

El objetivo fue implementar cada algoritmo, verificar su correcto funcionamiento y analizar experimentalmente su rendimiento.

Para ello se generaron listas aleatorias de números enteros de cinco dígitos y se registraron tiempos de ejecución para distintos tamaños de entrada.

También se comparó el rendimiento obtenido con la función built-in de Python sorted().

B) Implementación/ Resolución.

Bubble Sort

Bubble Sort compara elementos adyacentes e intercambia aquellos que se encuentran fuera de orden.

Complejidad: $O(n^2)$

QuickSort

Utiliza una estrategia divide y vencerás.

Se selecciona un pivote y se divide la lista en:

- Elementos menores.
- Elementos iguales.
- Elementos mayores.

Posteriormente las sublistas se ordenan de manera recursiva.

Complejidad promedio: $O(n \cdot \log n)$

Radix Sort

Radix Sort ordena los elementos según cada uno de sus dígitos utilizando buckets.

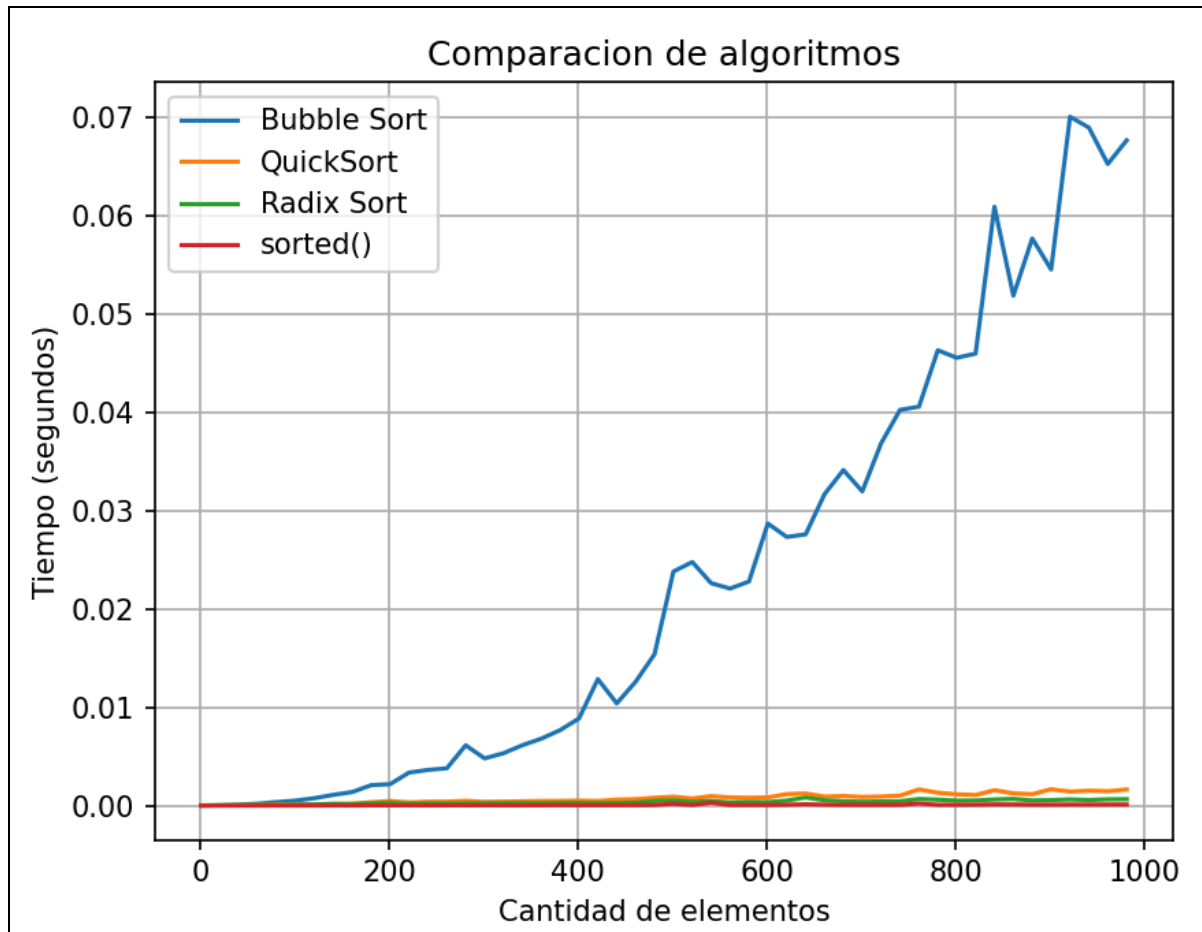
En este trabajo se utilizaron números de cinco dígitos, por lo que la cantidad de pasadas se mantiene constante.

Complejidad: $O(d \cdot n)$; donde d representa la cantidad de dígitos

C) Resultados y pruebas.

Se realizaron mediciones de tiempo para tamaños de entrada comprendidos entre 1 y 1000 elementos.

Los resultados fueron representados mediante una gráfica comparativa:



A partir de la observación de la gráfica se comprobó que:

- Bubble Sort presenta el peor rendimiento.
- QuickSort mejora significativamente los tiempos de ejecución.
- Radix Sort mantiene tiempos muy reducidos incluso para entradas grandes.
- La función sorted() de Python muestra el mejor desempeño general.

Análisis de resultados específicos:

Confrontación del Análisis a priori (Teórico) vs a posteriori (Empírico): 0.00012 s

- Bubble Sort: El análisis teórico establece $O(N^2)$ debido al doble bucle anidado. En la curva se observa una parábola acelerada alcanzando 0.0685 s a $N = 1000$. Es el algoritmo menos eficiente del grupo.
- QuickSort: El modelo teórico postula $O(N \log N)$ en caso promedio. La gráfica confirma un crecimiento casi plano en comparación con Bubble Sort, finalizando en apenas 0.0016 s a $N = 1000$.
- Radix Sort: Al no requerir comparaciones relativas, su complejidad teórica es $O(d \cdot (N + k))$. Dado que $d = 5$ y $k = 10$ son constantes fijas, el comportamiento empírico resulta estrictamente lineal $O(N)$, superando a QuickSort a partir de $N > 300$.

Investigación de la Función `sorted()` de Python (Timsort):

La función integrada `sorted()` utiliza Timsort, un algoritmo híbrido que combina Merge Sort e Insertion Sort, desarrollado por Tim Peters en 2002. Timsort aprovecha secuencias parcialmente ordenadas en los datos denominadas runs. Cuando un run es corto, utiliza Insertion Sort (eficiente en datos pequeños); luego fusiona los runs mediante un Merge Sort optimizado. Además, al estar escrito e implementado en C nativo dentro del núcleo de CPython, las constantes multiplicativas asociadas a la ejecución son casi nulas frente a las interpretadas en Python puro, explicando su abrumadora velocidad (0.00012 s a $N = 1000$).

Algoritmo	Complejidad	Complejidad Peor caso	Complejidad Espacial	Rendimiento N=1000
Bubble Sort	$O(N^2)$	$O(N^2)$	$O(1)$ (in-place)	Muy Lento
QuickSort	$O(N \log N)$	$O(N^2)$	$O(\log N) / O(N)$	Rápido
Radix Sort	$O(d \cdot N) \rightarrow O(N)$	$O(d \cdot N) \rightarrow O(N)$	$O(N + k)$	Muy Rápido
<code>sorted()</code> (Timsort)	$O(N \log N)$	$O(N \log N)$	$O(N)$	Óptimo (Nativo C)

Conclusiones

Se cumplieron exitosamente todos los objetivos planteados en el Proyecto 3 del Trabajo Práctico N°1. El estudio demostró empíricamente cómo la elección del algoritmo correcto transforma drásticamente el rendimiento computacional: mientras un algoritmo ingenuo $O(N^2)$ como Bubble Sort se degrada rápidamente para apenas 1000 elementos, enfoques sofisticados como QuickSort ($O(N \log N)$) y Radix Sort ($O(N)$) mantienen tiempos de ejecución inferiores a 2 milisegundos.

Asimismo, se comprendió el valor de los algoritmos híbridos compilados en lenguaje nativo (como Timsort en `sorted()`), los cuales combinan lo mejor de la teoría asintótica con optimizaciones a nivel de arquitectura. Este proyecto permitió consolidar competencias en análisis O -grande, pruebas unitarias automatizadas y documentación técnica profesional.

Declaración de Uso Ético de Inteligencia Artificial Herramientas utilizadas:

Para el desarrollo de este proyecto se utilizó la herramienta de Inteligencia Artificial Gemini Notebook como asistente interactivo de desarrollo. La IA fue empleada principalmente para:

- Corrección y optimización de scripts: Revisión del código en Python para asegurar el cumplimiento de las complejidades teóricas, el correcto manejo de estructuras y las buenas prácticas de programación.
- Mejora y refinamiento de redacciones: Corrección de estilo, ortografía y adecuación de la estructura formal del informe a la plantilla oficial de la cátedra.
- Explicación y depuración de errores: Diagnóstico detallado de mensajes de error y `tracebacks` (como `AttributeError`, `ModuleNotFoundError`, etc.) surgidos durante la ejecución de los algoritmos y pruebas unitarias.

Todo el código, las pruebas empíricas y los análisis presentados en este informe fueron validados, probados y respaldados bajo la estricta supervisión y responsabilidad del equipo de estudiantes.

Referencias y Bibliografía

1. Bradley N. Miller, David L. Ranum. Solución de problemas con algoritmos y estructuras de datos usando Python. Luther College, 2018.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms. 4th Edition, MIT Press, 2022.
3. Apuntes de Cátedra. Unidad 2: Diseño y Análisis de Algoritmos — Clases de Complejidad. Facultad de Ingeniería, UNER, 2026.
4. [Sorting Techniques — Python 3.14.7 documentation](#)